

Placement Portal Application

Project Report: Modern Application Development I
IIT Madras Online Degree Programme

1. Author

Full Name	Priyanshi Dalwadi
Roll Number	24F3001002
Email	24f3001002@ds.study.iitm.ac.in
About	I’m a dual degree student (BTech CSE), interested in understanding how systems work at a deeper level and building solutions that are actually useful, not just theoretical. This project is an attempt to apply what I’ve learned in a more practical, hands-on way and make the concepts feel concrete.

2. AI / LLM Usage

AI tools were used at various stages of this project. Claude (Anthropic) was used to help debug SQLAlchemy queries and assist with writing parts of this report. DiagramGPT (eraser.io) was used to generate the ER diagram of the database.

Once the core functionality - models, routes, and CRUD flows for all three roles was complete and working, a separate UI pass was carried out: Claude was used to design and generate a shared base.html layout and CSS design system (colour palette, typography, and reusable components), which was then applied across every existing template. All AI-generated content was reviewed and adapted manually and database design, route logic, and overall project structure were independently decided and implemented, and the visual direction, page content, and per-template integration of the design system were reviewed and adjusted by hand.

Tools used: Claude (Anthropic), DiagramGPT by eraser.io

Category	Backend (Flask + DB)	Frontend (HTML/CSS)	Styling / UI	Auth + CRUD Logic	Extras (Diagrams etc.)
AI Usage	12-17%	15-25%	40-60%	5-8%	35%
Notes	Used for debugging SQLAlchemy queries and route structure	Used for Jinja2 template structure and form layouts, mostly syntax help	Added after core completion — Claude drafted the base.html design system (CSS tokens, typography, shared components); applied and adjusted by hand across all templates	Logic independently written; AI used only for syntax help	DiagramGPT used for ER diagram generation

3. Project Description

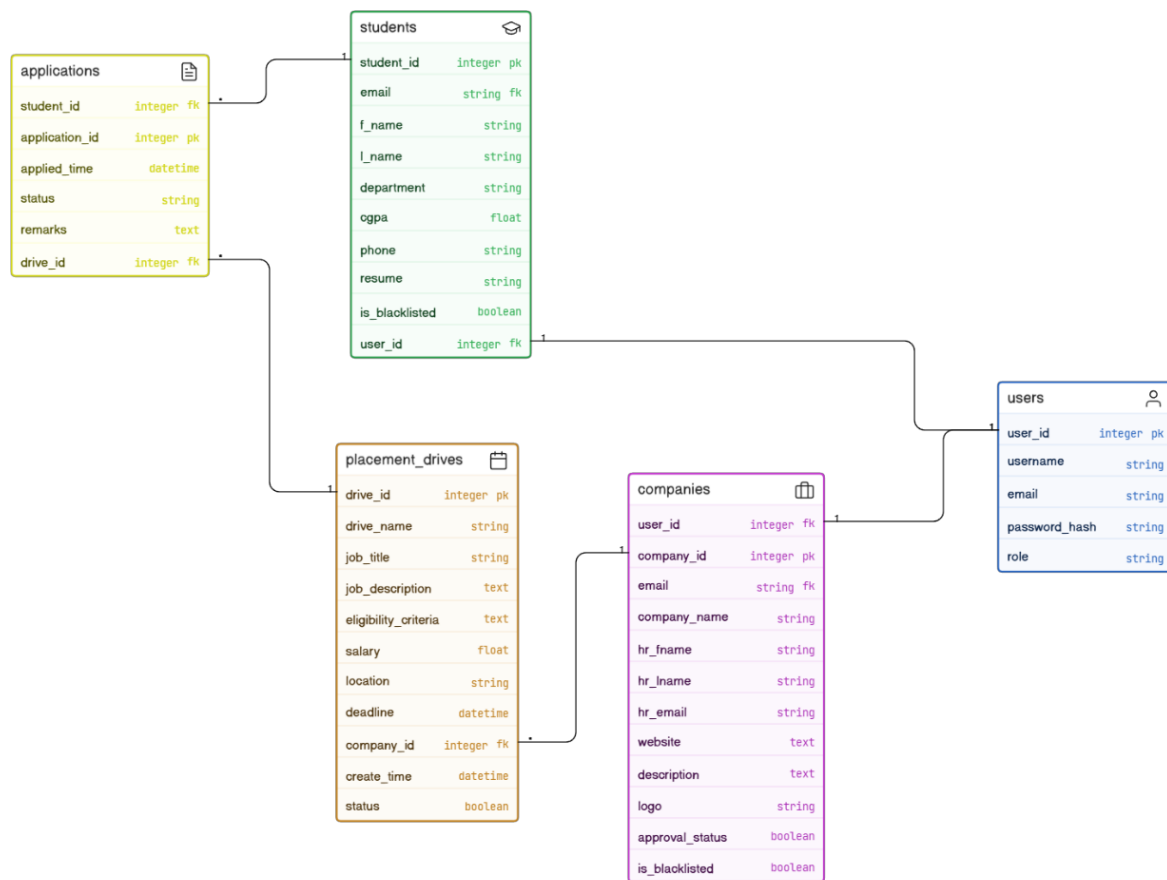
This project is a role-based campus placement portal built using Flask. It allows three types of users: Admin (Institute), Companies, and Students to interact with the system according to their roles.

- Companies can register, post placement drives, and manage applications.
- Students can browse active drives and apply to them.
- The Admin oversees the entire system: approving companies, managing drives, and monitoring student and application data.

4. Technologies Used

Technology	Purpose
Python 3 / Flask	Core web framework - handles routing, request/response lifecycle, and blueprint-based modular structure
Flask-SQLAlchemy	ORM layer for database interactions; maps Python model classes to SQLite tables
SQLite	Lightweight file-based relational database; suitable for development and local deployment
Werkzeug	Password hashing (generate_password_hash / check_password_hash) and secure filename handling for uploads
Jinja2	Server-side HTML templating; renders dynamic content in all views
Flask Sessions	Stateless server-side sessions used to track logged-in user identity and role across requests
HTML / CSS	Frontend presentation layer via Jinja2 templates, organised by role (auth, admin, company, student)
CSS Design System (base.html)	Shared visual layer added after core completion - CSS custom properties for colour and typography, Google Fonts (Playfair Display, IBM Plex Sans/Mono), and reusable UI components (topbar, stat cards, data tables, badges, forms) extended by every template

5. Database Schema Design



Entity-relationship diagram - generated with DiagramGPT (eraser.io)

6. UI / Frontend Design System

Once the database schema, models, routes, and CRUD flows for all three roles were complete and working, a unified front-end design pass was applied across the entire application. Every template — all ~24 files across auth/, admin/, company/, and student/ - was rewritten to extend a single shared layout, templates/base.html. This was a presentation-layer change only: no routes, models, or business logic were modified as part of this pass.

base.html defines a small set of design tokens as CSS custom properties (--ink, --paper, --amber, --sage, --rust, --font-display, --font-body, --font-mono, --r, etc.), which every page then reuses. The visual identity is a dark, editorial theme - a near-black background, warm off-white text, and an amber accent used for highlights, calls to action, and the active navigation state, with sage and rust used for success and error states. Typography combines Playfair Display for headings, IBM Plex Sans for body copy, and IBM Plex Mono for labels, navigation, and badges, all loaded from Google Fonts.

base.html also provides the shared chrome and components used on every page:

- A sticky topbar with role-aware navigation links and a colour-coded role badge (admin, company, student)

- A flash-message region wired to Flask's `get_flashed_messages`, styled per category (success / error / warning / info)
- Page headers with an oversized background "watermark" word for each section
- Stat grids and cards for dashboard summaries (e.g. total students, companies, drives)
- Data tables, status badges (waiting / shortlist / reject, approved / pending / blacklisted), and form styling shared across admin, company, and student views
- A hero/landing layout with role-selection cards for the home page, and profile-hero blocks with key-value lists for student and company profiles
- Empty-state placeholders for lists with no data, and small entry animations on page load

Individual templates now only define a `{% block body %}` (and, where needed, a `{% block extra_head %}` for page-specific styles or animations); everything else - layout, fonts, navigation, and the design tokens - is inherited from `base.html`.

7. Architecture and Features

Architecture

```

placement-portal-app/
├── app.py
├── models.py
├── extensions.py
├── routes/
│   ├── auth.py
│   ├── admin.py
│   ├── company.py
│   └── student.py
├── templates/
│   ├── base.html
│   ├── auth/
│   ├── admin/
│   ├── company/
│   └── student/
└── static/
    └── uploads/
        ├── resumes/
        └── logos/
  
```

The project follows a standard MVC-style Flask application structure. `app.py` is the entry point: it creates the Flask application, registers four Blueprints (auth, admin, company, student), initialises SQLAlchemy via `extensions.py`, and seeds a default admin user on first run. Business logic and route handlers live in the `routes/` directory, one file per role. Database models are defined in `models.py` using SQLAlchemy declarative syntax. All HTML views are Jinja2 templates stored under `templates/`, sharing a common `base.html` layout and grouped by role (`auth/`, `admin/`, `company/`, `student/`). Static files - uploaded resumes and logos - are stored under `static/uploads/`.

Default Features

- Role-based authentication with session management (admin, company, student)
- Company registration with admin approval gate before login is permitted
- Placement drive lifecycle: create → active → closed (manual or automatic on deadline)
- Student application flow: browse drives → apply → view status (waiting / shortlist / reject)

- Admin dashboard showing aggregate stats: total students, approved companies, drives, and placed count
- File uploads: PDF resumes for students, PNG/JPG logos for companies
- Search and filter on admin management views (companies, drives, students)

Additional Features

- Automatic drive expiry: on every dashboard load, drives past their deadline are automatically marked as closed
- Blacklisting: admin can toggle blacklist status on both companies and students to restrict access
- Company ownership enforcement: companies can only edit or close their own drives; unauthorised attempts are rejected with a flash message
- Resume access control: student resumes are viewable by the company reviewing the application and by the admin
- Application remarks: companies can leave private notes alongside shortlist/reject decisions

8. Video

https://drive.google.com/drive/folders/1cJucKnLy5rA_CxLSY2bgCZ2gmqoZg8uf?usp=sharing